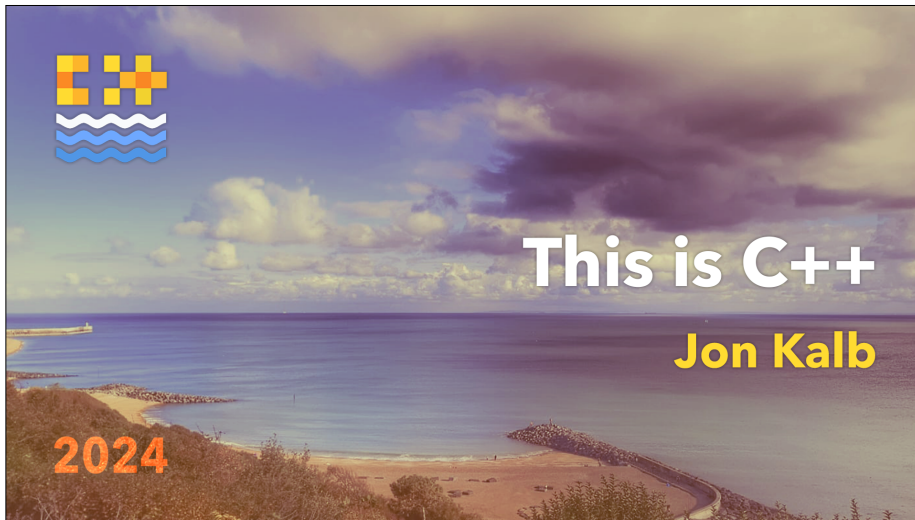
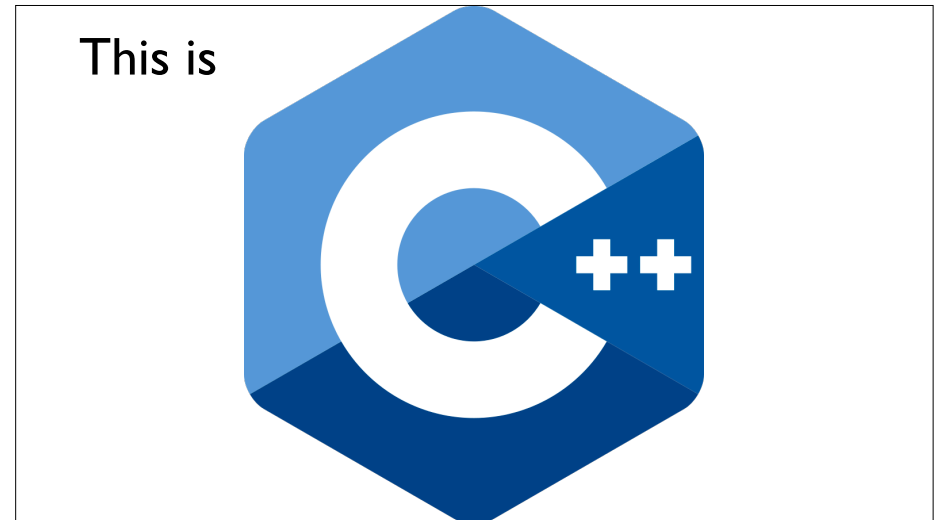
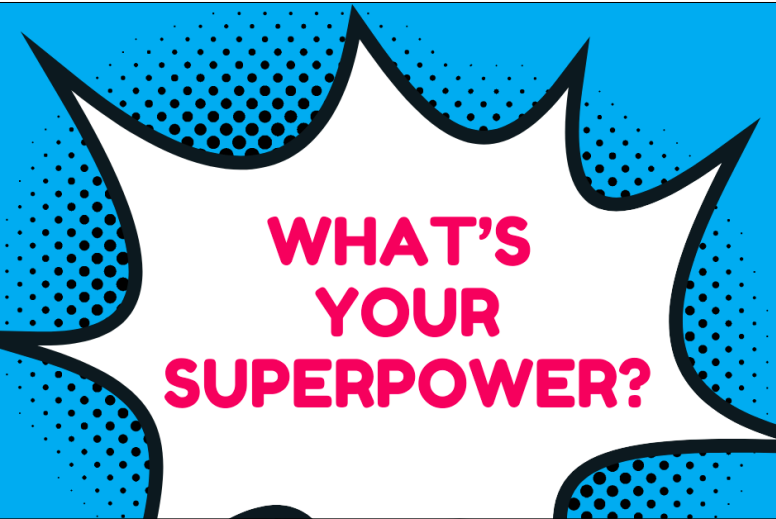




1



2

A comic book style speech bubble with a jagged black outline. The interior of the bubble is white, and the background outside the bubble is blue with a black halftone dot pattern.

**WHAT'S  
YOUR  
SUPERPOWER?**

3

A comic book style speech bubble with a jagged black outline. The interior of the bubble is white, and the background outside the bubble is blue with a black halftone dot pattern.

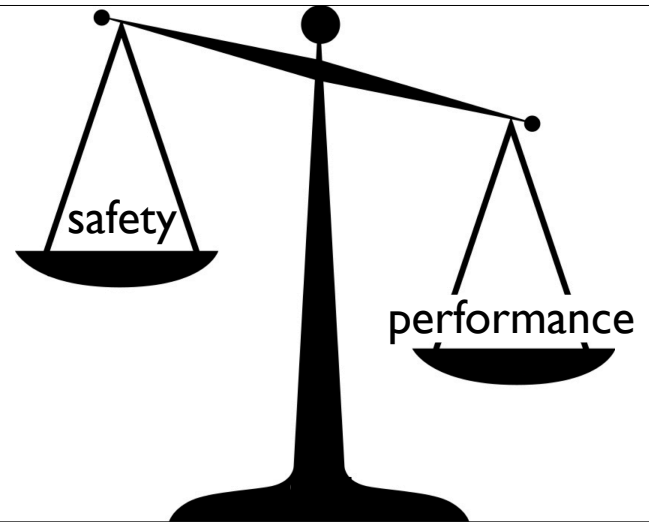
**Uncompromising  
performance!**

4

This is



5



6

This is



7

*“leave no room for a lower level language”*

— P2137R0 (Originally Bjarne Stroustrup)

8

## | Zero Overhead Rule

- ◆ if a feature is not used: no cost
- ◆ if a feature is used: no greater cost

9

9

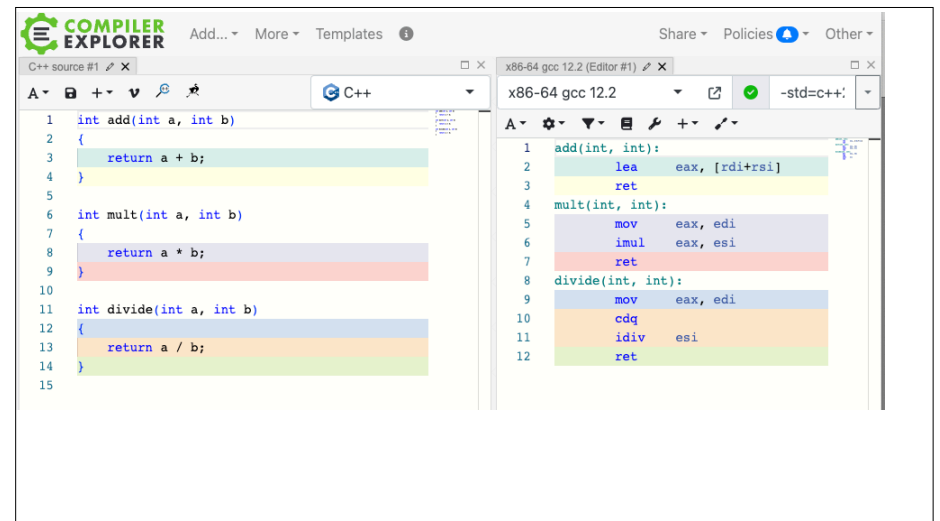


How do we  
do it?

10

the user's code is  
implemented directly  
with corresponding  
machine instructions  
(if they exist)

11

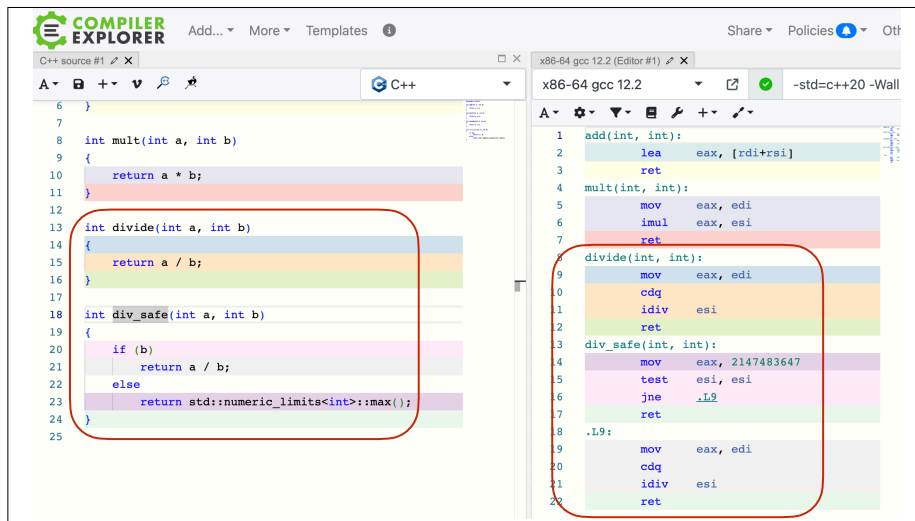


The screenshot shows the Compiler Explorer interface. The left pane displays C++ source code for three functions: `add`, `mult`, and `divide`. The right pane shows the corresponding assembly code generated by x86-64 gcc 12.2. The assembly code for `add` is a single `leaq` instruction. The assembly code for `mult` uses `mov`, `imul`, and `ret`. The assembly code for `divide` uses `mov`, `cdq`, `idiv`, and `ret`.

```
1 int add(int a, int b)
2 {
3     return a + b;
4 }
5
6 int mult(int a, int b)
7 {
8     return a * b;
9 }
10
11 int divide(int a, int b)
12 {
13     return a / b;
14 }
15
```

```
1 add(int, int):
2     leaq    eax, [rdi+rsi]
3     ret
4 mult(int, int):
5     mov     eax, edi
6     imul    eax, esi
7     ret
8 divide(int, int):
9     mov     eax, edi
10    cdq
11    idiv     esi
12    ret
```

12



```
6 }
7
8 int mult(int a, int b)
9 {
10     return a * b;
11 }
12
13 int divide(int a, int b)
14 {
15     return a / b;
16 }
17
18 int div_safe(int a, int b)
19 {
20     if (b)
21         return a / b;
22     else
23         return std::numeric_limits<int>::max();
24 }
25
```

```
1 add(int, int):
2     lea     eax, [rdi+rsi]
3     ret
4 mult(int, int):
5     mov     eax, edi
6     imul    eax, esi
7     ret
8 divide(int, int):
9     mov     eax, edi
10    cdq
11    idiv     esi
12    ret
13 div_safe(int, int):
14    mov     eax, 2147483647
15    test     esi, esi
16    jne     .L9
17    ret
18 .L9:
19    mov     eax, edi
20    cdq
21    idiv     esi
22    ret
```

13

## divide\_const

performance cost

◆ twice as many instructions

◆ includes a branch

note:

◆ simplest action on *divide by zero*  
(return a constant)

14

14

This is



15

| `std::vector`

both

◆ `operator[]()` `<==` efficient (not checked)

◆ `at()` `<==` safe (range checked)

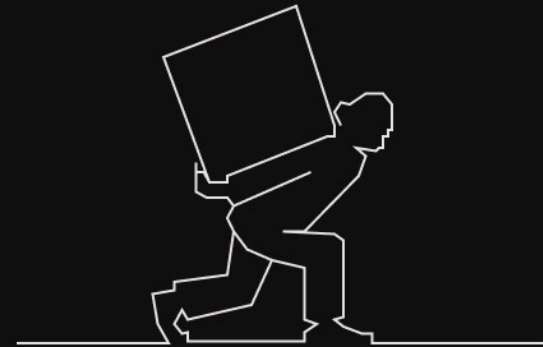
16

16

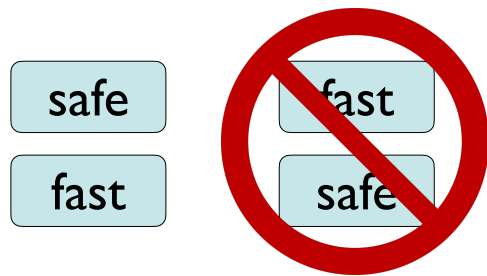
This is



17



18



19

This is



20

Undefined Behavior!

21

This is



22

Erroneous behavior for  
uninitialized reads

P2795R5 (C++26) - Thomas Köppe

```
int a; // not zero initialized
```

This is



25

```
int a{void}; // hypothetical syntax
```

26

```
int a{void}; // hypothetical syntax
```

```
int a = a;   // legal today
```

```
int a; // hypothetical future syntax  
      // for zero initialization
```

This is



29

```
int a;    // not initialized
```

30

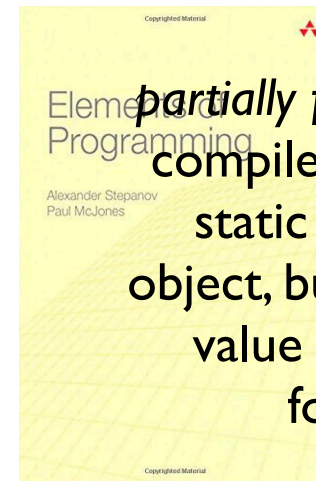
```
int a;    // not initialized  
int b{a}; // ??
```

31

```
int a;    // not initialized  
int b{a}; // undefined behavior
```

32

```
auto double_in_place(int& v)
{
    v *= 2;
}
```



*partially formed object:*  
compiler knows the  
static type of an  
object, but its runtime  
value is not fully  
formed

This is



35



36

This is



37

Undefined  
Behavior!

38

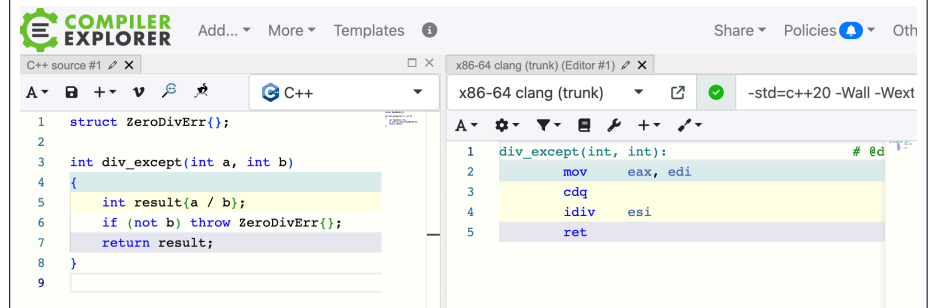
```

int div_except(int a, int b)
{
    int result{a / b};
    if (not b) throw ZeroDiv{};
    return result;
}

```

Will this ever be  
thrown?

39



The screenshot shows the Compiler Explorer interface. The left pane displays the C++ source code, and the right pane shows the generated assembly code for the x86-64 clang (trunk) compiler.

```

C++ source #1
1 struct ZeroDivErr();
2
3 int div_except(int a, int b)
4 {
5     int result{a / b};
6     if (not b) throw ZeroDivErr();
7     return result;
8 }
9

x86-64 clang (trunk) (Editor #1)
x86-64 clang (trunk) -std=c++20 -Wall -Wext
1 div_except(int, int):
2     mov     eax, edi
3     cdq
4     idiv    esi
5     ret

```

40

*Is C++ safe?*

41



**Modern C++:**  
move semantics  
rvalue reference syntax

42

This is



43

```
widget::widget(  
    std::list<Gadget> && inGL  
    ):  
    mGadgets{std::move(inGL)}  
{}
```

44

cppreference.com

Page Discussion

C++ Containers library std::vector

```
std::vector<T,Allocator>::vector  
constexpr vector( vector&& other ) noexcept; (since C++20)
```

vector move is  
*noexcept*

cppreference.com

Page Discussion

C++ Containers library std::list

```
std::list<T,Allocator>::list  
list( list&& other ); (8) (since C++11)
```

list move is  
**not** *noexcept*

45

## move enabling

- ◆ primitive type ==> shallow copy
- ◆ (owning) pointer ==> shallow copy, zero the source
- ◆ compound type ==> (create and) use move operations that recursively follow these rules
- ◆ destructor: must be safe on moved from objects
- ◆ assignment operator: must be safe to assign to moved from objects

46

46

allocation: <some address>  
size: 10  
capacity: 16

allocation: <nil>  
size: 10  
capacity: 16

*“...a sentinel node is a specifically designated node used with linked lists and trees as a traversal path terminator. This type of node does not hold or reference any data managed by the data structure.”*

— *Wikipedia*



49

*What do we call invariants that don't  
always hold?*

*They are called **not invariants**.*

50

# Sutter's Mill

Herb Sutter on software development

## Move, simply

 Herb Sutter  2020-02-17  9 Minutes

51

Table 33: *Cpp17MoveAssignable* requirements [\[tab:cpp17.moveassignable\]](#)

| Expression                                                                                                                                                                                                                                                                                                                        | Return type         | Return value   | Post-condition                                                                                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>t = rv</code>                                                                                                                                                                                                                                                                                                               | <code>T&amp;</code> | <code>t</code> | If <code>t</code> and <code>rv</code> do not refer to the same object, <code>t</code> is equivalent to the value of <code>rv</code> before the assignment |
| <code>rv</code> 's state is unspecified.                                                                                                                                                                                                                                                                                          |                     |                |                                                                                                                                                           |
| <i>[Note 2:</i> <code>rv</code> must still meet the requirements of the library component that is using it, whether or not <code>t</code> and <code>rv</code> refer to the same object. The operations listed in those requirements must work as specified whether <code>rv</code> has been moved from or not. — <i>end note]</i> |                     |                |                                                                                                                                                           |

52

```

unique_ptr<int> a[]
{
    make_unique<int>(1),
    make_unique<int>(0)
};

sort(begin(a),
     end(a),
     [](const auto& a, const auto& b)
     { return *a < *b; });

```

UB if moved from

53

Table 33: *Cpp17MoveAssignable* requirements [\[tab:cpp17.moveassignable\]](#)

| Expression                                                                              | Return type         | Return value   | Post-condition                                                                                                                                            |
|-----------------------------------------------------------------------------------------|---------------------|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>t = rv</code>                                                                     | <code>T&amp;</code> | <code>t</code> | If <code>t</code> and <code>rv</code> do not refer to the same object, <code>t</code> is equivalent to the value of <code>rv</code> before the assignment |
| rv is assignable-to and destructible. [ Note:<br>rv's state is otherwise unspecified. ] |                     |                |                                                                                                                                                           |

54



**Warning:  
std::find() is broken!**

SEAN PARENT



20  
21 |   
October 24-29

55

## Relaxing Requirements of Moved-From Objects

P2345 - Sean Parent

56

*What is the meaning of these function signatures?*

```
auto foo(widget& w);
```

```
auto foo(widget&& w);
```

The blog's  
suggestion:  
these two  
mean the  
same.

57

```
ViewPortID CreateViewPort(/**/);  
Void FreeViewPt(ViewPortID vpID);  
int GetBitDepth(ViewPortID vpID);  
int GetXDim(ViewPortID vpID);  
...
```

58

```

struct ViewPort
{
    ViewPort(**/):
        vpID_(CreateViewPt(**/))
        {/**RAII*/}

    ~ViewPort(){if (vpID_)
                FreeViewPt(vpID_);}

    ~~~~
private:
    ViewPortID vpID_;
};

```

How do we  
implement move  
operations?

59

```

struct ViewPort
{
    ~~~~
    ViewPort(ViewPort&& vp):
        vpID_(vp.vpID_)
        {vp.vpID_ = 0;}
    ~~~~
};

```

60

```

struct ViewPort
{
    ~~~~
    ViewPort(ViewPort&& vp):
        vpID_(vp.vpID_)
        {vp.vpID_ = 0;
        /* Call OS to make a
        new ViewPort */}
    ~~~~
};

```

61

```

struct ViewPort
{
    ~~~~
    ViewPort(ViewPort&& vp):
        vpID_(vp.vpID_)
        {vp.vpID_ = 0;}
    /* modify all member functions
    to "fake" something when moved
    From */
    ~~~~
};

```

62

## | Summary

- ◆ What is C++?
  - ◆ Uncompromised performance.
- ◆ Achieved by: living with UB, accepting a safety trade-off

63

63

## | Summary (continued)

- ◆ Committee: Standard types: moved from == fully formed
- ◆ This ship has sailed
- ◆ IMHO: a mistake because:
  - ◆ Minor/major performance hits
  - ◆ Leads to logic errors

64

64

## | Summary (continued)

- ◆ Committee direction: user code moved from == fully formed
- ◆ Minor reduction in cognitive load, but encourages logic errors.
- ◆ Some use cases, unacceptable performance, defeating move semantics.

65

65

## | Summary (continued)

- ◆ Correct:  
moved from == partially formed

66

66

This is



67

| Summary (continued)

◆ Application safety != language safety

68

68

This is

